Plateforme intelligente de pilotage et d'analyse décisionnelle du CIAT

Guide d'installation et de déploiement

ISEP Amadou Trawaré de Diamniadio · Centre d'Innovation, d'Application et de Transfert
Évaluation pratique en intelligence artificielle générative · Projet 3 · document généré le 03/11/2026

Guide d'installation, de configuration et de déploiement

1. Prérequis

Outil	Version testée	Remarque
Python	3.12 à 3.14	Python 3.11 minimum
Node.js	20 ou plus (testé avec 26)	Pour le frontend
PostgreSQL	16	Local, Docker, ou hébergé
Git	—	Pour le déploiement

2. Base de données PostgreSQL (au choix)

Option A – Docker (le plus simple) :

```
docker compose up -d
# DATABASE_URL=postgresql+psycopg://ciat:ciat_local@127.0.0.1:5432/ciat_vision
```

La base de test `ciat_vision_test` est créée automatiquement.

Option B – PostgreSQL installé : créer un utilisateur et deux bases :

```
CREATE USER ciat WITH PASSWORD 'choisir-un-mot-de-passe';
CREATE DATABASE ciat_vision OWNER ciat;
CREATE DATABASE ciat_vision_test OWNER ciat;
```

Option C – PostgreSQL portable sans installation (utilisée pour le développement, Docker étant indisponible) :

```
mkdir .outils && cd .outils && echo '{"name":"outils","private":true}' > package.json
npm install @embedded-postgres/windows-x64@16.15.0-beta.19     # linux-x64 / darwin-arm64 selon le système
cd ..
BIN=.outils/node_modules/@embedded-postgres/windows-x64/native/bin
echo "mot-de-passe-local" > .outils/pwfile
$BIN/initdb -D .pgdata -U postgres --pwfile=.outils/pwfile -E UTF8 --auth=scram-sha-256
$BIN/pg_ctl -D .pgdata -o "-p 5433" -l .outils/postgres.log start      # arrêt : pg_ctl -D .pgdata stop
```

Puis créer les bases `ciat_vision` et `ciat_vision_test` (par exemple avec un client SQL ou Python/psycopg).

3. Backend (API FastAPI)

```
cd backend
python -m venv .venv
.venv\Scripts\activate            # Linux / macOS : source .venv/bin/activate
pip install -r requirements-dev.txt
copy .env.example .env            # Linux / macOS : cp .env.example .env  → puis compléter
alembic upgrade head              # crée les 16 tables, contraintes et index
uvicorn app.main:app --reload     # http://127.0.0.1:8000  ·  documentation de l'API : /docs
```

Variables d'environnement (`backend/.env`) :

Variable	Obligatoire	Rôle
`DATABASE_URL`	oui	Connexion PostgreSQL (`postgres://…` est aussi accepté)
`JWT_SECRET`	oui	Clé de signature des jetons, 32 caractères aléatoires minimum : `python -c "import secrets; print(secrets.token_urlsafe(48))"`
`CORS_ORIGINS`	oui	Adresse(s) du frontend, séparées par des virgules
`ANTHROPIC_API_KEY`	non	Active la rédaction des réponses par l'IA (Claude). Sans clé : mode dégradé
`CREATE_DEMO_ACCOUNTS`	non	`true` : crée admin / coordo / analyste si aucun compte n'existe
`SEED_SIMULATED_DATA`	non	`true` : charge les données SIMULÉES si la base ne contient aucun projet

Au premier démarrage : domaines, comptes de démonstration et données simulées sont créés automatiquement.

Identifiant	Mot de passe	Rôle
`admin`	`Admin@2026`	Administrateur
`coordo`	`Coordo@2026`	Coordonnateur du CIAT
`analyste`	`Analyste@2026`	Analyste

⚠️ Changer ces mots de passe (page Profil) ou mettre `CREATE_DEMO_ACCOUNTS=false` avant toute utilisation réelle.

Commandes utiles :

```
python -m app.seed --reset                # purge puis régénère les données simulées
python -m scripts.generer_dictionnaire    # régénère docs/03_base_de_donnees.md
python -m pytest                          # 80 tests sur la base TEST_DATABASE_URL (fichier .env)
```

4. Frontend (React)

```
cd frontend
npm install
npm run dev        # http://localhost:5173 (les appels /api sont redirigés vers le port 8000)
npm test           # tests Vitest
npm run build      # version de production dans frontend/dist
```

Déploiement en un seul service : si `frontend/dist` existe, l'API FastAPI sert aussi le frontend. Après `npm run build`, l'application complète est disponible sur http://127.0.0.1:8000.

5. Déploiement sur Render

1. Publier le dossier `CIAT_VISION` dans un dépôt GitHub (privé de préférence).
2. Sur https://render.com : **New** → **Blueprint**, choisir le dépôt. Le fichier `render.yaml` crée : - la base `ciat-vision-db` (PostgreSQL) ; - le service `ciat-vision-api` (migrations appliquées à chaque démarrage, puis uvicorn) ; - le site statique `ciat-vision` (frontend).
3. Après la première création, reporter les adresses réelles attribuées par Render : - `CORS_ORIGINS` du service API = adresse du site statique ; - `VITE_API_URL` du site statique = adresse de l'API ; puis relancer un déploiement du site.
4. Facultatif : saisir `ANTHROPIC_API_KEY` dans *Environment* du service API.
5. Tester `https://<api>/api/health`, puis se connecter sur le site.

Points d'attention (offres gratuites) : - le service est mis en veille après une période d'inactivité : le premier chargement peut prendre une minute. **Ouvrir l'application quelques minutes avant la présentation au jury** ; - les bases gratuites ont une durée de vie et un volume limités : consulter les conditions de Render et prévoir une sauvegarde (`pg_dump`, voir § 7).

6. Passage aux données réelles du CIAT

1. Se connecter en administrateur → **Administration** → **Données** → saisir `PURGER` → **Purger**. Les données simulées sont supprimées ; comptes, domaines et paramètres sont conservés.
2. **Importation** : télécharger les modèles, puis importer dans l'ordre : cohortes → projets → financements.
3. Mettre `SEED_SIMULATED_DATA=false` et `CREATE_DEMO_ACCOUNTS=false` sur le serveur.
4. Le bandeau « Données simulées » disparaît automatiquement.

7. Sauvegarde et restauration

```
pg_dump --format=custom --file=ciat_vision_$(date +%F).dump "$DATABASE_URL"
pg_restore --clean --if-exists --dbname="$DATABASE_URL" ciat_vision_AAAA-MM-JJ.dump
```

8. Logo

Le logo officiel est intégré (versions décrites dans `docs/04_maquettes.md` § 2). Pour le changer plus tard, remplacer `docs/logo/ciat_vision_logo_original.png`, lancer `python scripts/preparer_logo.py` (fond rendu transparent et versions recadrées automatiquement ; ajuster les zones de recadrage du script si la mise en page du nouveau logo diffère), puis reconstruire le frontend (`npm run build`). Aucune modification du code n'est nécessaire.

9. Sécurité en production – liste de contrôle

- [ ] `JWT_SECRET` aléatoire et unique, jamais commité.
- [ ] Comptes de démonstration désactivés ou mots de passe changés.
- [ ] `CORS_ORIGINS` limité à l'adresse du frontend.
- [ ] HTTPS (automatique sur Render).
- [ ] Sauvegardes planifiées de la base.
- [ ] Clé d'API de l'IA stockée uniquement dans les variables d'environnement du serveur.
- [ ] Examen de la déclaration à la CDP avant l'import de données personnelles réelles (loi n° 2008-12).